

Soluzioni elaborato di Calcolo Numerico

Docì Leandro

a.a. 2025/2026

Esercizio 1. Combinare linearmente $y(t+h)$, $y(t)$, $hy'(t)$, $hy'(t-h)$, $hy'(t-2h)$, con coefficienti non nulli e indipendenti da h , in modo da ottenere un residuo che sia $\mathcal{O}(h^4)$. Dettagliare il procedimento.

Soluzione. Vogliamo trovare i coefficienti $a, b, c, d, e \in \mathbb{R}$, $a, b, c, d, e \neq 0$ tali che:

$$ay(t+h) + by(t) + chy'(t) + dhy'(t-h) + ehy'(t-2h) = R(h)$$

tale che il residuo sia dell'ordine $\mathcal{O}(h^4)$.

Scriviamo lo sviluppo di Taylor di ciascun termine.

$$y(t+h) = y(t) + hy'(t) + \frac{h^2}{2}y''(t) + \frac{h^3}{6}y'''(t) + \frac{h^4}{24}y^{(4)}(t) + \mathcal{O}(h^5) \quad (1)$$

$$y(t) = y(t) \quad (2)$$

$$hy'(t) = hy'(t), \quad (3)$$

$$hy'(t-h) = h \left(y'(t) - hy''(t) + \frac{h^2}{2}y'''(t) - \frac{h^3}{6}y^{(4)}(t) + \mathcal{O}(h^5) \right), \quad (4)$$

$$hy'(t-2h) = h \left(y'(t) - 2hy''(t) + 2h^2y'''(t) - \frac{4h^3}{3}y^{(4)}(t) + \mathcal{O}(h^5) \right). \quad (5)$$

Sostituiamo nella combinazione lineare i risultati ottenuti e raggruppiamo i termini:

$$\begin{aligned} & y(t)(a+b) + hy'(t)(a+c+d+e) + h^2y''(t)\left(\frac{a}{2} - d - 2e\right) + \\ & + h^3y'''(t)\left(\frac{a}{6} + \frac{d}{2} + 2e\right) + h^4y^{(4)}(t)\left(\frac{a}{24} - \frac{d}{6} - \frac{4e}{3}\right) = R(h) \end{aligned}$$

Per fare in modo che il residuo sia $\mathcal{O}(h^4)$ bisogna annullare i coefficienti dei primi quattro termini. Otteniamo il sistema:

$$\begin{cases} a+b=0, \\ a+c+d+e=0, \\ \frac{a}{2} - d - 2e=0, \\ \frac{a}{6} + \frac{d}{2} + 2e=0. \end{cases}$$

Questo è un sistema con 5 incognite e 4 equazioni. È necessario quindi fissare arbitrariamente una incognita, scegliamo a . Otteniamo quindi che:

$$\begin{cases} b = -a \\ c = -a - d - e \\ d = \frac{a}{2} - 2e \\ e = -\frac{a}{12} - \frac{d}{4} \end{cases} \Rightarrow \begin{cases} b = -a \\ c = -a - d - e \\ d = \frac{a}{2} - 2e \\ e = -\frac{a}{12} - \frac{a}{8} + \frac{2e}{4} \end{cases} \Rightarrow \begin{cases} b = -a \\ c = -a - \frac{4a}{3} - \frac{5a}{12} = -\frac{23a}{12} \\ d = \frac{a}{2} + 2 \cdot \frac{5a}{12} = \frac{4a}{3} \\ e = -\frac{5a}{12} \end{cases}$$

La soluzione del sistema esiste quindi per $a \in \mathbb{R}, a \neq 0$

Verifichiamo che sia vero assegnando un valore $a = 12$:

$$\begin{cases} b = -12 \\ c = -\frac{23 \cdot 12}{12} = -23 \\ d = \frac{4 \cdot 12}{3} = 16 \\ e = -\frac{5 \cdot 12}{12} = -5 \end{cases}$$

Da cui otteniamo che:

$$\begin{aligned} & y(t)(a+b) + h y'(t)(a+c+d+e) + h^2 y''(t)\left(\frac{a}{2} - d - 2e\right) + \\ & + h^3 y'''(t)\left(\frac{a}{6} + \frac{d}{2} + 2e\right) + h^4 y^{(4)}(t)\left(\frac{a}{24} - \frac{d}{6} - \frac{4e}{3}\right) = \\ & 0y(t) + 0h y'(t) + 0h^2 y''(t) + 0h^3 y'''(t) + \frac{9}{2}h^4 y^{(4)}(t) = \frac{9}{2}h^4 y^{(4)}(t) = R(h) \end{aligned}$$

Il quale rispetta la condizione di residuo di ordine $O(h^4)$, pertanto la soluzione è valida.

Esercizio 2. Scrivere una function Matlab che, sapendo solamente che l'aritmetica finita implementata utilizza la base binaria con arrotondamento, calcoli: la precisione di macchina; il più grande numero di macchina; il più piccolo numero di macchina normalizzato; il più piccolo numero di macchina denormalizzato. Argomentare la stesura, che deve prevedere solo l'utilizzo delle operazioni algebriche elementari (validare i risultati con le function `realmax` e `realmin` di Matlab).

Soluzione: L'obiettivo è realizzare una function Matlab che, attraverso l'uso esclusivo di operazioni algebriche elementari, calcoli i parametri fondamentali di un'aritmetica finita in base binaria (b=2) con approssimazione per arrotondamento, coerentemente con lo standard IEEE-754

Precisione di macchina La precisione di macchina (u) è definita come una maggiorazione uniforme dell'errore relativo di rappresentazione per i numeri normalizzati. In un'aritmetica con base b e mantissa di m cifre con arrotondamento, essa vale:

$$u = \frac{1}{2}b^{1-m}$$

Nello standard IEEE 754 il suo valore è pari a $2^{-53} \approx 10^{-16}$. Operativamente, può essere definita come il più piccolo numero positivo tale che la sua somma con 1 sia distinguibile da 1 in aritmetica finita, ovvero tale che:

$$fl(1 + u) > 1$$

Sotto questo numero, se si effettua la somma non si riesce più a distinguere la differenza. Per trovare il valore è necessario quindi diminuire il valore della variabile di precisione fino a quando la somma con 1 non è più distinguibile.

Numero di macchina più grande normalizzato Il più grande numero rappresentabile, denotato come overflow limit (r_M), si ottiene quando la mantissa ha tutte le cifre al valore massimo ($b - 1$) e l'esponente è il massimo consentito. Oltre questo limite si verifica il fenomeno dell'overflow, e la macchina restituisce convenzionalmente ∞ .

Per individuarlo quindi è necessario trovare la potenza di 2 più grande rappresentabile, raddoppiando ogni volta finché si vede che il raddoppiamento successivo è infinito. Una volta fatto ciò si moltiplica la potenza con la mantissa più grande rappresentabile, pari a $(2 - u)$ con u precisione di macchina.

Numero di macchina più piccolo normalizzato Il minimo numero normalizzato (r_m) rappresenta il più piccolo numero positivo rappresentabile nella forma $1.f \cdot 2^{e-shift}$. Al di sotto di questo valore, l'accuratezza della rappresentazione diminuisce poiché non è più possibile mantenere la prima cifra della mantissa diversa da zero. Per ricavarlo quindi è necessario diminuire il valore della variabile del numero fino a quando non avviene il fenomeno di underflow, raggiungendo i numeri denormalizzati. Si ripristina la normalizzazione moltiplicando alla fine per 2.

Numero di macchina più piccolo denormalizzato Lo standard IEEE-754 implementa il gradual underflow, che permette di rappresentare numeri più piccoli di r_m utilizzando mantisse del tipo $0.f$. Il più piccolo numero denormalizzato è l'estremo limite prima dell'underflow assoluto, dove il valore viene arrotondato a zero. Per ricavarlo è semplicemente necessario trovare il numero più piccolo rappresentabile sopra lo 0.

```

1 function [prec, maxNum, minNor, minDen] = calcPrecision()
2 % [prec, maxNum, minNor, minDen] = calcPrecision()
3 % Funzione che calcola valori base di una aritmetica finita in
  base binaria
4 % con arrotondamento
5 % Output:
6 %   prec - precisione di macchina
7 %   maxNum - numero di macchina piu' grande
8 %   minNor - numero di macchina piu' piccolo normalizzato
9 %   minDen - numero di macchina piu' piccolo denormalizzato
10
11 % 1. Calcolo della precisione
12 prec = 1;
13 while (1.0 + prec / 2) > 1.0
14     prec = prec / 2;
15 end
16

```

```

17 % 2. Calcolo del numero di macchina piu' grande
18 maxNum = 1.0;
19 while isfinite(maxNum * 2)
20     maxNum = maxNum * 2;
21 end
22 maxNum = maxNum * (2.0 - prec);
23
24 % 3. Calcolo del numero di macchina piu' piccolo normalizzato
25 minNor = 1.0;
26 while (minNor/2.0) + (minNor/2.0) * prec > (minNor/2.0)
27     minNor = minNor / 2.0;
28 end
29
30 % 4. Calcolo del numero di macchina piu' piccolo denormalizzato
31 minDen = minNor;
32 while (minDen/2.0) > 0
33     minDen = minDen / 2.0;
34 end
35
36 fprintf('Validazione Risultati:\n');
37 fprintf('precisione: calcolato = %e, eps = %e\n', prec, eps);
38 fprintf('maxNum: calcolato = %e, realmax = %e\n', maxNum, realmax
    );
39 fprintf('minNor: calcolato = %e, realmin = %e\n', minNor, realmin
    );
40 fprintf('minDen: calcolato = %e, realmin*eps = %e\n', minDen,
    realmin*eps);
41 end

```

Esercizio 3. Scrivere una function Matlab che implementi il metodo di Newton per gli zeri di una funzione. Prevedere valori di default per la tolleranza e per il numero massimo di iterazioni.

Soluzione:

```

1 function x = newtonZeros(f, f1, x0, tol, maxit, molt)
2 % function x = newtonZeros(f, f1, x0, tol, maxit, molt)
3 % metodo per la ricerca di zeri di una funzione tramite metodo di
    Newton
4 % modificato
5 % Input:
6 %     f - function che implementa la funzione f
7 %     f1 - function che implementa la derivata della funzione f
8 %     x0 - punto iniziale
9 %     tol - tolleranza di arresto (default = 1e-12)
10 %     maxit - numero massimo di iterazioni (default = 1000)
11 %     molt - molteplicita' della radice (default = 1)
12 % Output:
13 %     x - approssimazione della radice
14

```

```

15 if nargin < 3, error('Numero insufficiente di parametri'), end
16 if nargin < 6, molt = 1; end
17 if nargin < 5, maxit = 1000; end
18 if nargin < 4, tol = 1e-12; end
19
20 if molt < 1 || molt ~= fix(molt), error('Molteplicita'' non
    valida'); end
21 if maxit <= 1, error('Numero di interazioni massime non valido');
    end
22 if tol <= 0, error('Tolleranza non valida'); end
23
24 x = x0;
25 flag = 1;
26 for i=1:maxit
27     x0 = x;
28     fx = feval(f, x);
29     f1x = feval(f1, x);
30     if fx == 0
31         flag = 0;
32         break;
33     end
34     if f1x == 0, error('Derivata nulla'); end
35     x = x0 - molt * (fx/f1x);
36     if abs(x-x0) <= (tol * (1 + abs(x)))
37         flag = 0;
38         break;
39     end
40 end
41 if flag, warning('Accuratezza non raggiunta'), end
42 end

```

Esercizio 4. Scrivere una function Matlab che implementi il metodo di bisezione. Prevedere valori di default per la tolleranza di arresto.

Soluzione:

```

1 function x = bisezione(f, a, b, tol)
2 % x = bisezione(f, a, b, tol)
3 % Metodo per la ricerca degli zeri di una funzione tramite metodo
    di bisezione
4 % Input:
5 %     f - function che implementa la funzione f
6 %     a, b - estremi dell'intervallo
7 %     tol - tolleranza di arresto (default 1e-12)
8 % Output:
9 %     x - approssimazione della radice
10
11 if nargin < 3, error('Numero di paramametri insufficienti'), end
12 if nargin < 4, tol = 1e-12; end
13

```

```

14 if a >= b, error('Estremi dell'' intervallo non validi'), end
15 if tol <= 0, error('Tolleranza non valida'), end
16
17 fa = feval(f, a);
18 fb = feval(f, b);
19 if fa == 0, x = a; return, end
20 if fb == 0, x = b; return, end
21 if fa * fb > 0, error('Non esiste radice della funzione nell''
    intervallo inserito'), end
22
23
24 maxit = ceil(log2(b-a) - log2(tol));
25
26 for i = 1:maxit
27     x = (a+b)/2;
28     fx = feval(f, x);
29     def = abs(fb-fa)/(b-a);
30     if abs(fx) <= tol * def
31         break;
32     elseif fa * fx < 0
33         b = x;
34         fb = fx;
35     else
36         a = x;
37         fa = fx;
38     end
39 end
40 end

```

Esercizio 5. Scrivere una function Matlab che implementi un metodo ibrido: inizialmente il metodo di bisezione e, non appena l'ampiezza dell'intervallo di confidenza diventi minore dell'ampiezza prefissata δ (e se non sia usciti in precedenza) passi ad utilizzare il metodo di Newton, a partire dall'ultima approssimazione trovata dal metodo di bisezione, fino ad ottenere la massima accuratezza possibile.

Soluzione:

```

1 function x = hybridSearch(f, f1, a, b, delta, tol, maxit)
2 % x = hybridSearch(f, f1, a, b, delta, tol, maxit)
3 % Metodo di ricerca degli zeri di una funzione tramite un
    algoritmo ibrido
4 % tra metodo di bisezione e metodo di Newton
5 % Input:
6 %     f - function che implementa la funzione f
7 %     f1 - function che implementa la derivata della funzione f
8 %     a, b - estremi dell'intervallo
9 %     delta - ampiezza dell'intervallo minima nel metodo di
    bisezione
10 %     (default 1e-1)
11 %     tol - tolleranza di arresto (default 1e-12)
12 %     maxit - numero massimo di iterazioni (default 1000)

```

```

13
14 if nargin < 4, error('Parametri insufficienti'), end
15 if nargin < 7, maxit = 1000, end
16 if nargin < 6, tol = 1e-12, end
17 if nargin < 5, delta = 1e-1, end
18 if maxit < 1, error('Numero massimo di iterazioni non valido'),
    end
19 if tol <= 0, error('Tolleranza non valida'), end
20 if delta <= 0, error('Delta non valido'), end
21 if a >= b, error(' Estremi dell''intervallo non validi'), end
22
23 fa = feval(f, a);
24 fb = feval(f, b);
25 if fa == 0, x = a; return, end
26 if fb == 0, x = b; return, end
27 if fa * fb > 0, error('Non esiste radice della funzione nell''
    intervallo inserito'), end
28
29 while b-a > delta
30     x = (a+b)/2;
31     fx = feval(f, x);
32     if fx == 0, break;
33     elseif fa * fx < 0
34         b = x;
35     else
36         a = x;
37         fa = fx;
38     end
39 end
40
41 x = (a+b)/2;
42 flag = 1;
43
44 for i = 1:maxit
45     x0 = x;
46     fx = feval(f, x);
47     f1x = feval(f1, x);
48     if fx == 0
49         flag = 0;
50         break;
51     end
52     if f1x == 0, error('Derivata nulla'); end
53     x = x0 - (fx/f1x);
54     if abs(x-x0) <= (tol * (1 + abs(x)))
55         flag = 0;
56         break;
57     end
58 end
59 if flag, warning('Accuratezza non raggiunta'), end
60 end

```

Esercizio 6. Tabulare il costo computazionale dei precedenti metodi in termini di valutazioni di funzione, per determinare lo zero della funzione

$$f(x) = \sin 10x + \arctan 10x$$

con la massima accuratezza possibile. Utilizzare $x_0 = 10$ per il metodo di Newton, e l'intervallo iniziale $[-10.1, 10]$ per il metodo di bisezione ed il metodo ibrido. Per quest'ultimo, utilizzare $\delta = 10^{-1}$ come valore di switch tra i due metodi. Commentare i risultati ottenuti.

Soluzione: Presa la funzione $f(x)$, è stato ricercato uno zero della funzione utilizzando le tre function Matlab create con le seguenti condizioni:

- metodo di Newton: punto iniziale $x_0 = 10$;
- metodo di bisezione: intervallo $[-10.1, 10]$;
- metodo ibrido (bisezione + Newton): intervallo $[-10.1, 10]$ con soglia di switch $\delta = 10^{-1}$.

Calcolata la derivata di $f(x)$:

$$f'(x) = 10 \cos(10x) + \frac{10}{1 + 100x^2}$$

si vede che la funzione possiede un'unica radice reale semplice in $x = 0$:

$$f(0) = \sin 0 + \arctan 0 = 0, \quad f'(0) = 10 \cos 0 + \frac{10}{1 + 0} = 20 \neq 0,$$

I parametri rimanenti avranno il seguente valore:

`maxit = 1000`, `tol = 1e-15`, `molt = 1` poichè la radice è semplice.

Metodo	Converge	Radice	Iterazioni	f	f'	Totale
Newton	no	-5.6187×10^3	1000	1000	1000	2000
Bisezione	sì	-5.5263×10^{-17}	54	56	0	56
Ibrido	sì	0	8 + 4	14	4	18

I risultati ottenuti permettono di trarre le seguenti conclusioni:

Metodo di Newton: Il metodo di Newton ha la caratteristica di avere una convergenza locale, a differenza del metodo di bisezione. Questo è importante per il risultato ottenuto perchè il punto iniziale selezionato, $x_0 = 10$, è molto lontano dalla radice. Pertanto la funzione cade in una regione in cui la derivata oscilla rapidamente, fermandosi all'esaurimento del numero di iterazioni e mostrando un messaggio di avvertimento sull'accuratezza non raggiunta.

Il costo è il più alto: 1000 valutazioni di f più 1000 di f' (Newton è l'unico che richiede anche la derivata), per un totale di 2000 valutazioni. La tolleranza richiesta è irrilevante, perchè la radice non viene individuata.

Metodo di bisezione: Poichè $f(-10.1) \cdot f(10) < 0$, per il teorema degli zeri esiste una radice nell'intervallo e la bisezione vi converge sempre, perchè la convergenza è globale.

Data la convergenza lineare è possibile ricavare a priori il numero massimo di iterazioni necessario:

$$\text{maxit}_{\text{teorico}} = \lceil \log_2(20.1) - \log_2(\text{tol}) \rceil = 55$$

A differenza del metodo di Newton viene eseguita una singola valutazione per iterazione oltre alle due valutazioni iniziali agli estremi. Il costo quindi è pari a 56 valutazioni di f (54 dalle iterazioni, 2 dalla valutazione degli estremi iniziale). La radice $(-5.5 \times 10^{-17}, |f| \approx 1.1 \times 10^{-15})$ è accurata alla precisione di macchina.

Metodo Ibrido: Il metodo combina i pregi dei due precedenti.

- **Fase 1 (bisezione):** dimezza l'intervallo finché la sua ampiezza scende sotto $\delta = 0.1$. In questo caso sono sufficienti 8 dimezzamenti per localizzare la radice in un intorno sufficiente per la convergenza locale del metodo di Newton.
- **Fase 2 (Newton):** avendo trovato precedentemente un intorno alla radice, partendo dal punto medio, si sfrutta la convergenza quadratica per trovare la radice in un numero di iterazioni minore.

Il metodo quindi unisce i punti di forza di entrambi i metodi: tramite il metodo di Bisezione si risolve il problema della convergenza locale del Metodo di Newton trovando un intorno della radice, invece con il metodo di Newton si sfrutta la convergenza quadratica per ottenere una radice ad un costo inferiore. Il costo del metodo è il seguente: 14 valutazioni per $f(x)$ e 4 valutazione per $f(x)'$, per un totale quindi di 18 valutazioni.

Il numero di iterazioni eseguite sono invece 8 nella parte di bisezione e 4 per la parte di Newton. Questo mostra un netto vantaggio nell'utilizzo di un algoritmo combinato rispetto all'utilizzo dei metodi in modo autonomo.

Esercizio 7. Scrivere una function Matlab, con sintassi `x = triup(U, b)` che risolva un sistema triangolare superiore $n \times n$, in cui la matrice dei coefficienti U è memorizzata, per colonne, in un vettore di lunghezza $n(n+1)/2$.

Soluzione:

```
1 function x = triup(U, b)
2 % x = triup(U, b)
3 % Metodo di risoluzione di un sistema triangolare superiore
4 % Input:
5 %   U - vettore rappresentante la matrice triangolare superiore
        dei coefficienti
6 %   b - vettore dei termini noti
7 % Output:
8 %   x - vettore soluzione del sistema
9
10 if nargin < 2, error('Numero parametri insufficiente'), end
11 n = (-1 + sqrt(1 + 8 * length(U))) / 2;
12 if mod(n, 1) ~= 0, error('Lunghezza vettore della matrice dei
        coefficienti non valida'), end
13 n = round(n);
14 if n ~= length(b), error('Le dimensioni dei vettori non sono
        valide'), end
```

```

15
16 U = U(:);
17 x = b(:);
18 for i = n:-1:1
19     idxDiag = i*(i+1)/2;
20     if U(idxDiag) == 0, error('Matrice singolare'), end
21     x(i) = x(i) / U(idxDiag);
22     if i > 1
23         idxStart = (i-1)*i/2 + 1;
24         idxEnd = idxDiag - 1;
25         x(1:i-1) = x(1:i-1) - U(idxStart:idxEnd) * x(i);
26     end
27 end
28 end

```

Esercizio 8. Scrivi una function che implementi efficientemente la risoluzione di un sistema lineare mediante la fattorizzazione LU con pivoting parziale. Validare la function su alcuni esempi significativi.

Soluzione:

```

1 function x = LUPivotingSolve(A, b)
2 % x = LUPivotingSolve(A, b)
3 % Metodo di risoluzione di sistema lineare tramite
4   fattorizzazione LU con
5   % pivoting
6   % Input:
7   %   A - matrice dei coefficienti
8   %   b - vettore dei termini noti
9   % Output:
10  %   x - vettore soluzione del sistema
11
12 if nargin < 2, error('Numero di parametri insufficiente'), end
13 [m, n] = size(A);
14 if m ~= n || m ~= length(b), error('Dimensioni dei parametri non
15   validi'), end
16
17 P = (1:n);
18 % Fattorizzazione LU con Pivoting parziale
19 for i = 1:n-1
20     [mi, ki] = max(abs(A(i:n, i)));
21     if mi == 0, error('Matrice singolare'), end
22     ki = ki + i - 1;
23     if ki > i
24         A([i ki], :) = A([ki i], :);
25         P([i ki]) = P([ki i]);
26     end
27     A(i+1:n, i) = A(i+1:n, i) / A(i, i);
28     A(i+1:n, i+1:n) = A(i+1:n, i+1:n) - A(i+1:n, i) * A(i, i+1:n);
29 end

```

```

27 end
28
29 if A(n, n) == 0, error('Matrice singolare'), end
30 x = b(P);
31
32 % Risoluzione del sistema per L
33 for i = 2:n
34     x(i:n) = x(i:n) - A(i:n, i-1) * x(i-1);
35 end
36
37 % Risoluzione del sistema per U
38 for i = n:-1:1
39     x(i) = x(i) / A(i, i);
40     x(1:i-1) = x(1:i-1) - A(1:i-1, i) * x(i);
41 end
42 end

```

Per verificare la correttezza della function sono stati eseguiti una serie di test di diversa natura:

Test A – Sistema 3×3 .

$$A = \begin{pmatrix} 1 & 1 & 1 \\ 0 & 2 & 5 \\ 2 & 5 & -1 \end{pmatrix}, \quad b = \begin{pmatrix} 2 \\ 11 \\ -11 \end{pmatrix}.$$

La soluzione ottenuta è $x = \begin{pmatrix} 1 \\ -2 \\ 3 \end{pmatrix}$.

Test B – Pivoting necessario al primo passo.

$$A = \begin{pmatrix} 0 & 2 \\ 3 & 1 \end{pmatrix}, \quad b = \begin{pmatrix} 1 \\ 12.5 \end{pmatrix}.$$

Con $A_{11} = 0$, la fattorizzazione LU senza pivoting sarebbe impossibile poichè la matrice è singolare. In questo caso invece viene fatto lo scambio con ($A_{21} = 3$), ottenendo la soluzione esatta $x = \begin{pmatrix} 4 \\ 0.5 \end{pmatrix}$.

Test C – Sistema 4×4 .

$$A = \begin{pmatrix} 1 & 2 & 0 & -1 \\ 2 & 1 & 3 & 0 \\ -1 & 0 & 2 & 1 \\ 0 & 3 & 1 & 4 \end{pmatrix}, \quad b = \begin{pmatrix} 1 \\ 13 \\ 9 \\ 25 \end{pmatrix}.$$

La soluzione ottenuta è $x = \begin{pmatrix} 1 \\ 2 \\ 3 \\ 4 \end{pmatrix}$, in linea con l'effettiva soluzione del sistema.

Test D – Matrice tridiagonale simmetrica.

$$A = \begin{pmatrix} 2 & -1 & 0 \\ -1 & 2 & -1 \\ 0 & -1 & 2 \end{pmatrix}, \quad b = \begin{pmatrix} 1 \\ 0 \\ 5 \end{pmatrix}.$$

La matrice è simmetrica e a diagonale dominante in senso debole. La soluzione ottenuta è $x = \begin{pmatrix} 2 \\ 3 \\ 4 \end{pmatrix}$

Nel complesso, i quattro test confermano che la function implementa correttamente la fattorizzazione LU con pivoting parziale, riuscendo a gestire anche casi dove la fattorizzazione LU normalmente non darebbe un risultato corretto.

Esercizio 9. Al seguente link [drive link](#) è reperibile la function `linsis.m` che crea la matrice dei coefficienti ed il termine noto di un sistema lineare $n \times n$, la cui soluzione è il vettore con tutte le componenti uguali ad 1. Utilizzare la function del precedente esercizio per risolvere i sistemi lineari ottenuti con le seguenti istruzioni:

```
n = 10;
for k = 1:15
    [A,b] = linsis(n,k);
end
```

Rappresentare convenientemente l'andamento dell'errore, rispetto a k , e spiegare esaurientemente i risultati ottenuti.

Soluzione:

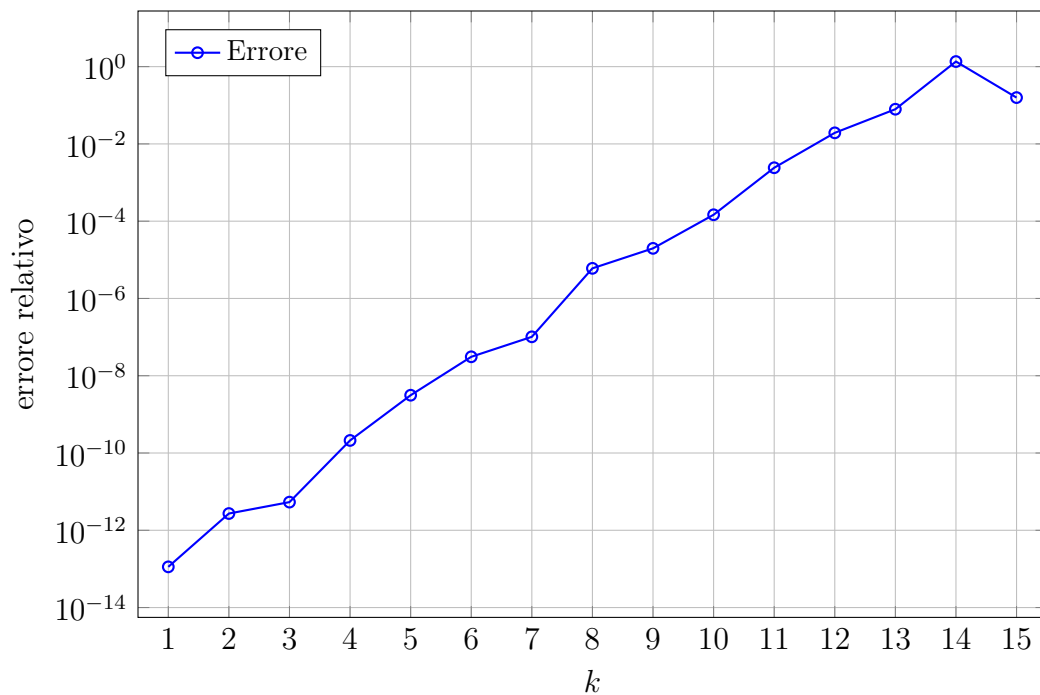
```
1 function [x, err] = errorLUP()
2 % [x, err] = errorLUP()
3 % Metodo di verifica dell'errore della fattorizzazione LU con
  pivoting.
4 % Genera 15 matrici A e vettori b e risolve i relativi sistemi
  lineari.
5 % Output:
6 %   x - vettore soluzione
7 %   err: errore commesso
8
9 n = 10;
10 y = ones(n, 1);
11
12 for k = 1:15
13     [A,b] = linsis(n,k);
14
15     x = LUPivotingSolve(A, b);
16     err = norm(x-y)/norm(y);
17     c = cond(A);
18     fprintf('Risultati sistema lineare numero %d: errore = %e,
  condizionamento = %e\n', k, err, c);
```

```
19 end
20 end
```

I risultati ottenuti dall'esecuzione del codice sono i seguenti:

Numero	Errore	Condizionamento
1	1.130752×10^{-13}	9.277376×10^3
2	2.712800×10^{-12}	9.108212×10^4
3	5.355708×10^{-12}	8.988955×10^5
4	2.114635×10^{-10}	8.871738×10^6
5	3.130597×10^{-9}	8.756054×10^7
6	3.089777×10^{-8}	8.641879×10^8
7	1.016495×10^{-7}	8.529199×10^9
8	6.019459×10^{-6}	8.417977×10^{10}
9	1.980370×10^{-5}	8.308105×10^{11}
10	1.466304×10^{-4}	8.201847×10^{12}
11	2.415982×10^{-3}	8.155220×10^{13}
12	1.933967×10^{-2}	8.476812×10^{14}
13	7.905092×10^{-2}	1.320487×10^{16}
14	1.355159	7.459594×10^{15}
15	1.581018×10^{-1}	5.061867×10^{16}

Dai risultati possiamo notare una correlazione tra l'indice k e il condizionamento della matrice. Al crescere dell'indice vediamo che le matrici diventano sempre più malcondizionate. Graficamente possono essere rappresentati nel grafico, che avvalorata come l'errore salga al crescere dell'indice, evidenziando la crescita del malcondizionamento.



Esercizio 10. Scrivere una function che implementi efficientemente la risoluzione di un sistema lineare, con matrice sdp, mediante la fattorizzazione LDL^T . Validare la function su alcuni esempi significativi.

Soluzione:

```
1 function x = LDLSolve(A, b)
2 % x = LDLSolve(A, b)
3 % Metodo di risoluzione di un sistema lineare con fattorizzazione
  LDL^T
4 % Input:
5 %   A - Matrice simmetrica definita positiva dei coefficienti
6 %   b - vettore dei termini noti
7 % Output:
8 %   x - vettore soluzione del sistema
9 if nargin < 2, error('Parametri insufficienti. '), end
10 [m, n] = size(A);
11 if m ~= n || m ~= length(b), error('Dimensione dei parametri non
    validi'), end
12
13 % Fattorizzazione
14 if A(1, 1) <= 0, error('Matrice non sdp'), end
15 A(2:n, 1) = A(2:n, 1) / A(1, 1);
16
17 for i = 2:n
18     v = (A(i,1:i-1).') .* diag(A(1:i-1, 1:i-1));
19     A(i, i) = A(i, i) - A(i, 1:i-1) * v;
20     if A(i,i) <= 0, error('Matrice non simmetrica definita
        positiva'), end
21     A(i+1:n, i) = (A(i+1:n, i) - A(i+1:n, 1:i-1) * v) / A(i,i);
22 end
23
24 % Risoluzione sistema Ax = b
25 x = b;
26 for i = 1:n
27     x(i+1:n) = x(i+1:n) - A(i+1:n, i) * x(i);
28 end
29 x = x ./ diag(A);
30
31 for i = n:-1:2
32     x(1:i-1) = x(1:i-1) - A(i, 1:i-1).'*x(i);
33 end
34 end
```

Validazione della function

I test effettuati per valutare la correttezza della function LDLSolve sono i seguenti:

Test A – Sistema 2×2 base.

$$A = \begin{pmatrix} 4 & 2 \\ 2 & 5 \end{pmatrix}, \quad b = \begin{pmatrix} -1 \\ -6.5 \end{pmatrix}.$$

La matrice è simmetrica e definita positiva. La soluzione ottenuta è $x = \begin{pmatrix} 0.5 \\ -1.5 \end{pmatrix}$

Test B – Matrice tridiagonale 3×3 .

$$A = \begin{pmatrix} 4 & -1 & 0 \\ -1 & 4 & -1 \\ 0 & -1 & 4 \end{pmatrix}, \quad b = \begin{pmatrix} 11 \\ -15 \\ 7 \end{pmatrix}.$$

Matrice simmetrica a diagonale dominante, dunque definita positiva. La soluzione ottenuta è $x = \begin{pmatrix} 2 \\ -3 \\ 1 \end{pmatrix}$.

Test C – Matrice prossima alla singolarità.

$$A = \begin{pmatrix} 1 & 0.99 \\ 0.99 & 1 \end{pmatrix}, \quad b = \begin{pmatrix} 5.05 \\ 4.9 \end{pmatrix}.$$

La matrice resta definita positiva ma con un numero di condizionamento non trascurabile, $\kappa(A) \approx 199$. La soluzione ottenuta è $x = \begin{pmatrix} 10 \\ -5 \end{pmatrix}$

Test D – Sistema 4×4 .

$$A = \begin{pmatrix} 10 & 1 & 2 & 3 \\ 1 & 10 & 1 & 2 \\ 2 & 1 & 10 & 1 \\ 3 & 2 & 1 & 10 \end{pmatrix}, \quad b = \begin{pmatrix} -2 \\ 24 \\ -26 \\ 38 \end{pmatrix}.$$

Matrice simmetrica a diagonale fortemente dominante. La soluzione ottenuta è $x = \begin{pmatrix} -1, \\ 2, \\ -3, \\ 4 \end{pmatrix}$.

Test E – Matrice non definita positiva.

$$A = \begin{pmatrix} 1 & 2 \\ 2 & 4 \end{pmatrix}, \quad b = \begin{pmatrix} 3 \\ 6 \end{pmatrix}.$$

La matrice A è simmetrica ma singolare, dunque non definita positiva. La function intercetta correttamente la violazione dell'ipotesi di positività e restituisce l'errore **'Matrice non simmetrica definita positiva'**, senza produrre alcun risultato numerico fuorviante.

Nel complesso, i cinque test confermano che la function implementa correttamente la fattorizzazione LDL^T : produce soluzioni accurate su matrici simmetriche definite positive di diversa struttura e dimensione, ed intercetta correttamente, tramite il controllo sul segno dei pivot durante la fattorizzazione, i casi in cui l'ipotesi di positività non è soddisfatta.

Esercizio 11. Utilizzando la funzione `linsis.m` su menzionata, utilizzare la funzione `function` del precedente esercizio per risolvere i sistemi lineari, con matrice `sd`, ottenuti con le seguenti istruzioni:

```
n = 10;
for k = 1:15
    [A,b] = linsis(n,k);
end
```

Rappresentare convenientemente l'andamento dell'errore, rispetto ad `k`, e spiegare esaurientemente i risultati ottenuti.

Soluzione:

```
1 function [x, err] = errorLDL()
2 % [x, err] = errorLDL()
3 % Metodo di verifica dell'errore della fattorizzazione LDL.
4 % Genera 15 matrici A e vettori b e risolve i relativi sistemi
   lineari.
5 % Output:
6 %   x - vettore soluzione
7 %   err: errore commesso
8
9 n = 10;
10 y = ones(n, 1);
11
12 for k = 1:15
13     [A,b] = linsis(n,k, 1);
14
15     x = LDLSolve(A, b);
16     err = norm(x-y)/norm(y);
17     c = cond(A);
18     fprintf('Risultati sistema lineare numero %d: errore = %e,
   condizionamento = %e\n', k, err, c);
19 end
20 end
```

I risultati ottenuti dall'esecuzione del codice sono i seguenti:

Numero	Errore	Condizionamento
1	3.909499×10^{-16}	9.869604
2	7.288680×10^{-16}	9.740909×10^1
3	5.121082×10^{-14}	9.613892×10^2
4	1.020972×10^{-13}	9.488531×10^3
5	3.614156×10^{-12}	9.364805×10^4
6	1.207156×10^{-11}	9.242692×10^5
7	1.305124×10^{-10}	9.122171×10^6
8	2.506725×10^{-9}	9.003222×10^7
9	2.073128×10^{-8}	8.885824×10^8
10	4.529523×10^{-8}	8.769954×10^9
11	2.363697×10^{-7}	8.655578×10^{10}
12	2.165668×10^{-5}	8.543180×10^{11}
13	2.753000×10^{-5}	8.429557×10^{12}
14	1.861264×10^{-3}	8.337155×10^{13}
15	1.840086×10^{-2}	8.570324×10^{14}

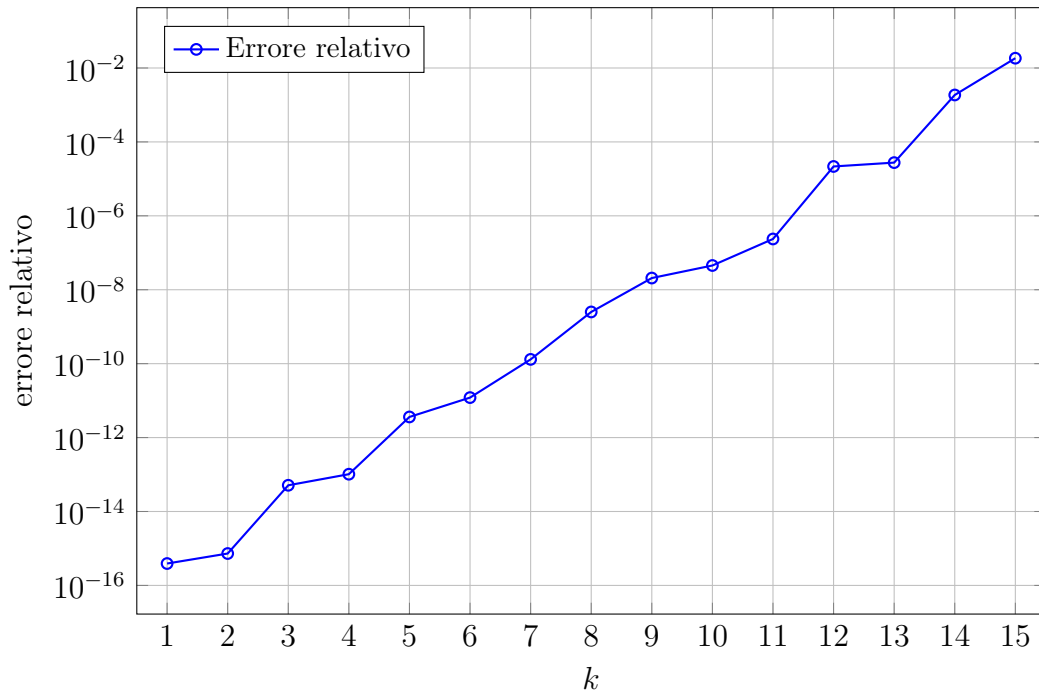


Figura 1: Andamento dell'errore relativo al crescere di k (scala logaritmica).

Dai risultati possiamo notare una correlazione tra l'indice k e il condizionamento della matrice. Al crescere dell'indice vediamo che le matrici diventano sempre più malcondizionate. Possiamo vedere però come il condizionamento sia migliore rispetto a quella della fattorizzazione LU con pivoting. Graficamente possono essere rappresentati come nel grafico, che avvalorava come l'errore salga al crescere dell'indice, evidenziando la crescita del malcondizionamento.

Esercizio 12. Dato un generico sistema lineare sovradeterminato a rango massimo

$$Ax = b, \quad A \in \mathbb{R}^{m \times n}, \quad m > n = \text{rank}(A),$$

ed indicato, al solito, con $r := Ax - b \in \mathbb{R}^m$ il corrispondente vettore residuo, scrivere una function Matlab che risolva il precedente problema nel senso dei minimi quadrati pesati. Ovvero minimizzando la seguente norma pesata:

$$\|r\|_{\omega}^2 = \sum \omega_i r_{i=1}^{2m}$$

dove $\omega_1, \dots, \omega_m$ sono pesi positivi assegnati.

Soluzione:

```

1 function [x, nor] = leastSquareSolve(A, b, omega)
2 % x = leastSquareSolve(A, b, omega)
3 % Metodo di risoluzione di un sistema lineare sovradeterminato
  con
4 % soluzione ai minimi quadrati pesati
5 % Input:
6 %   A - matrice sovradeterminata dei coefficienti
7 %   b - vettore dei termini noti
8 %   omega - vettore dei pesi positivi
9 % Output:
10 %   x - vettore soluzione del sistema
11 %   nor - norma del vettore residuo
12 if nargin < 3, error('Numero parametri insufficiente'), end
13 [m, n] = size(A);
14 if m <= n, error('Dimensione matrice non valida'), end
15 if m ~= length(b) || m ~= length(omega), error('Dimensione del
  vettore dei termini noti non valida'), end
16 if any(omega <= 0), error('I pesi non sono tutti positivi'), end
17
18 W = sqrt(omega(:));
19 A = A .* W;
20 b = b .* W;
21
22 for i = 1:n
23   alpha = norm(A(i:m, i));
24   if alpha == 0, error('Matrice non ha il rango massimo'), end
25   if A(i,i) >= 0, alpha = -alpha; end
26   v1 = A(i,i) - alpha;
27   A(i,i) = alpha;
28   A(i+1:m, i) = A(i+1:m, i)/v1;
29   beta = -v1/alpha;
30
31   v = [1;A(i+1:m, i)];
32   % applica H = I - beta*v*v' alle colonne successive
33   A(i:m, i+1:n) = A(i:m, i+1:n) - (beta * v * (v' * A(i:m, i+1:

```

```

n)));
34
35     % applica lo stesso riflettore a b
36     b(i:m) = b(i:m) - (beta * v * (v' * b(i:m)));
37 end
38
39 % Sostituzione all'indietro: R x = b(1:n)  (R triangolare sup. in
    A(1:n,1:n))
40 x = zeros(n, 1);
41 for i = n:-1:1
42     x(i) = (b(i) - A(i, i+1:n) * x(i+1:n)) / A(i,i);
43 end
44 nor = norm(b(n+1:m));
45 end

```

Esercizio 13. Siano assegnati:

$$A = \begin{pmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \\ 10 & 11 & 0 \end{pmatrix}, \quad b = \begin{pmatrix} 1 \\ 2 \\ 3 \\ 4 \end{pmatrix}.$$

Risolvere il sistema $Ax = b$ nel senso dei minimi quadrati pesati, cioè minimizzando (1) nei seguenti casi:

- $w_1 = 1, i = 1, \dots, 4$ (minimi quadrati standard)
- $w_1 = \frac{(5-i)^2}{9}, i = 1, \dots, 4$
- $w_1 = \frac{i^2}{9}, i = 1, \dots, 4$
- $w_1 = \sqrt{2}, i = 1, \dots, 4$

Riportare la norma (1) del corrispondente residuo per entrambi i casi. Commentare i risultati ottenuti.

Soluzione:

```

1 function x = executeLeastSquareSolve()
2 % x = executeLeastSquareSolve()
3 % Esecuzione della function di risoluzione di sistemi lineari
4 % sovradeterminati pesati
5 A = [1 2 3; 4 5 6; 7 8 9; 10 11 0];
6 b = [1; 2; 3; 4];
7 i = (1:4)';
8
9 omegas = {
10     ones(4,1);
11     ((5-i).^2)/9;
12     (i.^2)/9;
13     sqrt(2)*ones(4,1);

```

```

14 };
15
16 for k = 1:4
17     [x, nor] = leastSquareSolve(A, b, omegas{k});
18     fprintf('Caso %d:\n', k);
19     fprintf('    pesi      = [%s]\n', sprintf('%.4f ', omegas{k}));
20     fprintf('    x          = [%s]\n', sprintf('%.6f ', x));
21     fprintf('    ||r||_w = %.6e\n\n', nor);
22 end
23 end

```

Il sistema lineare $Ax = b$ è sovradeterminato, con $A \in \mathbb{R}^{4 \times 3}$ e $\text{rank}(A) = 3$, pertanto la soluzione ai minimi quadrati esiste ed è unica:

$$x = \begin{pmatrix} -\frac{1}{3} \\ \frac{2}{3} \\ 0 \end{pmatrix}$$

Dai risultati ottenuti mostrati sotto possiamo notare come indipendentemente dai pesi utilizzati, la soluzione trovata è la medesima. La differenza sta nel risultato della norma del vettore residuo. Il risultato della norma comunque è sempre un valore vicino allo zero.

Peso	Risultato	Norma residuo
$[1, 1, 1]^T$	$[-0.333333, 0.666667, -0.000000]^T$	$2.177593e - 16$
$[1.7778, 1.0000, 0.4444, 0.1111]^T$	$[-0.333333, 0.666667, -0.000000]^T$	$3.462501e - 17$
$[0.1111, 0.4444, 1.0000, 1.7778]^T$	$[-0.333333, 0.666667, -0.000000]^T$	$8.455176e - 16$
$[1.4142, 1.4142, 1.4142, 1.4142]^T$	$[-0.333333, 0.666667, -0.000000]^T$	$6.495059e - 16$

Esercizio 14. Scrivere una function che implementi efficientemente il metodo di Newton per risolvere sistemi di equazioni non lineari. Utilizzare le function sviluppate in precedenza come risolutori per i sistemi lineari coinvolti.

Soluzione:

```

1 function [x, iter] = nonLinearNewton(f, x0, tol, maxit)
2 % x = nonLinearNewton(f, x0, tol, maxit)
3 % Metodo di risoluzione di sistemi non lineari con metodo di
  Newton
4 % Input:
5 %   f - function che implementa la funzione f
6 %   x0 - approssimazione iniziale
7 %   tol - tolleranza (default 1e-12)
8 %   maxit - numero di iterazioni massime (default 1000)
9 % Output:
10 %   x - approssimazione della soluzione del sistema
11 %   iter - numero di iterazioni eseguite
12 if nargin < 3, error('Numero parametri insufficiente'), end
13 if nargin < 5, maxit = 1000; end

```

```

14 if nargin < 4, tol = 1e-12; end
15 if maxit < 1, error('Numero di iterazioni massime non valido'),
    end
16 if tol <= 0, error('Tolleranza non valida'), end
17
18 x = x0;
19 flag = 1;
20 iter = 0;
21
22 for i = 1:maxit
23     iter = iter + 1;
24     x0 = x;
25     [Fx, Jx] = feval(f, x);
26
27     delta = LUPivotingSolve(Jx, -Fx);
28     x = x0 + delta;
29     if norm(x-x0) <= tol * (1+norm(x))
30         flag = 0;
31         break;
32     end
33 end
34 if flag, warning('Accuratezza non raggiunta'), end
35 end

```

Esercizio 15. Utilizzare la precedente function per risolvere il sistema non lineare:

$$x_1^2 + x_2^2 = 5, \quad x_3^2 + x_4^4 = 5, \quad \cos(\pi x_1) + \cos(\pi x_3) = 0, \quad x_2^2 - x_4^2 = 0$$

Quante iterazioni sono necessarie per ottenere la massima accuratezza possibile, partendo dai punti iniziali:

$$x^0 = \begin{pmatrix} 0.4 \\ 2 \\ 0.4 \\ 2 \end{pmatrix}, \quad x^0 = \begin{pmatrix} 0.4 \\ -2 \\ -0.4 \\ 2 \end{pmatrix}, \quad x^0 = \begin{pmatrix} 1.4 \\ -2 \\ -1.4 \\ 2 \end{pmatrix}, \quad x^0 = \begin{pmatrix} 1.4 \\ 2 \\ 1.4 \\ 2 \end{pmatrix},$$

Riportare le soluzioni ottenute e commentare i risultati. Sarebbe possibile utilizzare il vettore nullo come punto iniziale (argomentare la risposta).

Soluzione:

```

1 function executeNonLinearNewton()
2
3 X0 = {
4     [0.4; 2; 0.4; 2],
5     [0.4; -2; -0.4; 2],
6     [1.4; -2; -1.4; 2],
7     [1.4; 2; 1.4; 2],
8 };
9

```

```

10 for k = 1:4
11     x0 = X0{k};
12     [x, iter] = nonLinearNewton(@systemFunction, x0, 1e-12, 1000)
13     ;
14     fprintf('Soluzione caso %d individuata in %d iterazioni \n',
15         k, iter);
16     disp(x);
17 end
18
19 fprintf('Tentativo con il vettore nullo:\n');
20
21 try
22     x = nonLinearNewton(@systemFunction, zeros(4,1), 1e-12, 1000)
23     ;
24     fprintf('Soluzione caso %d individuata in %d iterazioni \n',
25         k, iter);
26     disp(x);
27 catch ME
28     fprintf('Errore durante la risoluzione: %s\n', ME.message);
29 end
30 end
31
32 function [Fx,Jx] = systemFunction(x)
33
34 Fx = [
35     x(1)^2 + x(2)^2 - 5;
36     x(3)^2 + x(4)^4 - 5;
37     cos(pi*x(1)) + cos(pi*x(3));
38     x(2)^2 - x(4)^2
39 ];
40
41 Jx = [
42     2*x(1), 2*x(2), 0, 0;
43     0, 0, 2*x(3), 4*x(4)^3;
44     -pi*sin(pi*x(1)), 0, -pi*sin(pi*x(3)), 0;
45     0, 2*x(2), 0, -2*x(4)
46 ];
47
48 end

```

I risultati della function sono i seguenti: Possiamo notare che:

Appr. iniziale	Soluzione	num. Iterazioni
$[0.4, 2, 0.4, 2]^T$	$[1.7699, 1.3665, 1.2301, 1.3665]^T$	9
$[0.4, -2, -0.4, 2]^T$	$[1.7699, -1.3665, -1.2301, 1.3665]^T$	9
$[1.4, -2, -1.4, 2]^T$	$[1.7699, -1.3665, -1.2301, 1.3665]^T$	7
$[1.4, 2, 1.4, 2]^T$	$[1.7699, 1.3665, 1.2301, 1.3665]^T$	7

- con la prima e quarta approssimazione si ha la medesima soluzione, differenziando

nel numero di iterazioni svolte, mostrando che la quarta è geometricamente più vicina all'effettiva soluzione.

- con la seconda e terza approssimazione si ha la medesima soluzione, differenziando nel numero di iterazioni svolte, mostrando che la terza è geometricamente più vicina all'effettiva soluzione.

Utilizzando invece il vettore nullo $x^0 = \begin{pmatrix} 0 \\ 0 \\ 0 \\ 0 \end{pmatrix}$ si osserva che il risultato è un messaggio di

errore, in particolare viene detto

"Errore durante la risoluzione: Matrice singolare" Questo perchè andando a sostituire x si ottiene una matrice nulla che è singolare, perciò il metodo dà errore.

Esercizio 16. Scrivere una function Matlab

`y = lagrange(xi, fi, x)` in cui `(xi, fi)` contengono i dati del problema e `x` è un vettore di punti a cui corrispondono le valutazioni del polinomio interpolante in forma di Lagrange restituite nel vettore `y`.

Soluzione:

```
1 function y = lagrange(xi, fi, x)
2 % x = lagrange(xi, fi, x)
3 % Metodo di interpolazione polinomiale in forma di Lagrange
4 % Input:
5 %   xi - vettore delle ascisse di interpolazione
6 %   fi - vettore dei valori della funzione nei punti xi
7 %   x - vettore di punti di valutazione del polinomio
8 % Output:
9 %   y - valutazione del polinomio interpolante
10 if nargin < 3, error('Numero parametri insufficiente'), end
11 if length(xi) ~= length(fi), error('Vettori di lunghezze diverse'), end
12 if length(unique(xi)) ~= length(xi), error('Sono presenti ascisse non distinte'), end
13
14 n = length(xi);
15 y = zeros(size(x));
16
17 for i = 1:n
18     Li = ones(size(x));
19
20     for j = 1:n
21         if i ~= j
22             Li = Li .* (x - xi(j)) / (xi(i) - xi(j));
23         end
24     end
25     y = y + fi(i) * Li;
26 end
27 end
```

Esercizio 17. Scrivere una function Matlab

$y = \text{newton}(xi, fi, x)$ che ha la stessa sintassi e funzione della function del precedente esercizio, ma con l'utilizzo della forma di Newton del polinomio interpolante.

Soluzione:

```
1 function y = newton(xi, fi, x)
2 % y = newton(xi, fi, x)
3 % Metodo di interpolazione polinomiale in forma di Lagrange
4 % Input:
5 %   xi - vettore delle ascisse di interpolazione
6 %   fi - vettore dei valori della funzione nei punti xi
7 %   x - vettore di punti di valutazione del polinomio
8 % Output:
9 %   y - valutazione del polinomio interpolante
10 if nargin < 3, error('Numero parametri insufficiente'), end
11 if length(xi) ~= length(fi), error('Vettori di lunghezze diverse'), end
12 if length(unique(xi)) ~= length(xi), error('Sono presenti ascisse non distinte'), end
13
14 n = length(xi) - 1;
15 w = fi;
16
17 for j = 1:n
18     for i = (n + 1):-1:(j + 1)
19         w(i) = (w(i) - w(i-1)) / (xi(i) - xi(i-j));
20     end
21 end
22 y = w(n + 1) * ones(size(x));
23
24 for i = n:-1:1
25     y = y .* (x - xi(i)) + w(i);
26 end
27 end
```

Esercizio 18. Scrivere una function Matlab $[y, yi] = \text{hermite}(xi, fi, fi1, x)$ in cui $(xi, fi, fi1)$ contengono i dati del problema e x è un vettore di punti a cui corrispondono le valutazioni del polinomio interpolante di Hermite, e della sua derivata prima, rispettivamente restituite nei vettori y, yi .

Soluzione:

```
1 function [y, yi] = hermite(xi, fi, fi1, x)
2 % [y, yi] = hermite(xi, fi, fi1, x)
3 % Metodo di interpolazione polinoimiale tramite Hermite
```



```

4 % Input:
5 %   xi - vettore delle ascisse di interpolazione
6 %   fi - vettore dei valori della funzione nei punti xi
7 %   fi1 - vettore dei valori della derivata della funzione nei
      punti xi
8 %   x - vettore di punti di valutazione del polinomio
9 % Output:
10 %   y - valutazione del polinomio interpolante
11 %   yi - valutazioni della derivata prima
12 if nargin < 4, error('Parametri insufficienti'), end
13 if length(xi) ~= length(fi) || length(xi) ~= length(fi1), error('
      Vettori di lunghezze diverse'), end
14 if length(unique(xi)) ~= length(xi), error('Sono presenti ascisse
      non distinte'), end
15
16 n = length(xi);
17 m = 2*n;
18 z = zeros(m, 1);
19 Q = zeros(m, 1);
20
21 for i = 1:n
22     z(2*i-1) = xi(i);
23     z(2*i) = xi(i);
24     Q(2*i-1) = fi(i);
25     Q(2*i) = fi(i);
26 end
27
28 for i = m:-1:2
29     if mod(i, 2) == 0
30         Q(i) = fi1(i/2);
31     else
32         Q(i) = (Q(i)-Q(i-1)) / (z(i)-z(i-1));
33     end
34 end
35
36 for j = 2 : m-1
37     for i = m : -1 : j+1
38         Q(i) = (Q(i) - Q(i-1)) / (z(i) - z(i-j));
39     end
40 end
41
42 y = Q(m) * ones(size(x));
43 yi = zeros(size(x));
44
45 for i = m-1 : -1 : 1
46     yi = yi .* (x - z(i)) + y;
47     y = y .* (x - z(i)) + Q(i);
48 end
49
50 end

```

Esercizio 19. Scrivere una function Matlab `x = chebyshev(a, b, n)` che restituisce le ascisse di Chebyshev per il polinomio interpolante di grado `n` sull'intervallo `[a, b]`

Soluzione:

```

1 function x = chebyshev(a, b, n)
2 % x = chebyshev(a, b, n)
3 % Metodo di calcolo delle ascisse di Chebyshev per un polinomio
4 % interpolante di grado n
5 % Input:
6 %   a, b - estremi dell'intervallo di interpolazione
7 %   n - grado del polinomio interpolante (vengono generati n+1
      nodi)
8 % Output:
9 %   x - vettore dei nodi di Chebyshev sull'intervallo [a, b]
10 if nargin < 3, error('Numero parametri insufficiente'), end
11 if a >= b, error('Intervallo non valido'), end
12 if n ~= fix(n) || n < 0
13     error('Il grado n deve essere un intero non negativo.');

```

Esercizio 20. Graficare l'errore massimo (calcolato come il massimo di 10001 punti equispaziati nell'intervallo) per i polinomi interpolanti degli esercizi 16-18, relative alla funzione di Runge,

$$f(x) = (1 + x^2)^{-1}, \quad x \in [-10, 10],$$

utilizzando `n+1` ascisse equispaziate, con `n = 10, 20, 30, ..., 140` per i polinomi interpolanti in 16- 17, e `n = 10, 20, 30, ..., 70` per il polinomio interpolante di Hermite. Per quest'ultimo polinomio, graficare anche l'errore della derivata prima. Utilizzare il modo più appropriato per graficare i risultati. Commentare i risultati ottenuti.

Soluzione:

```

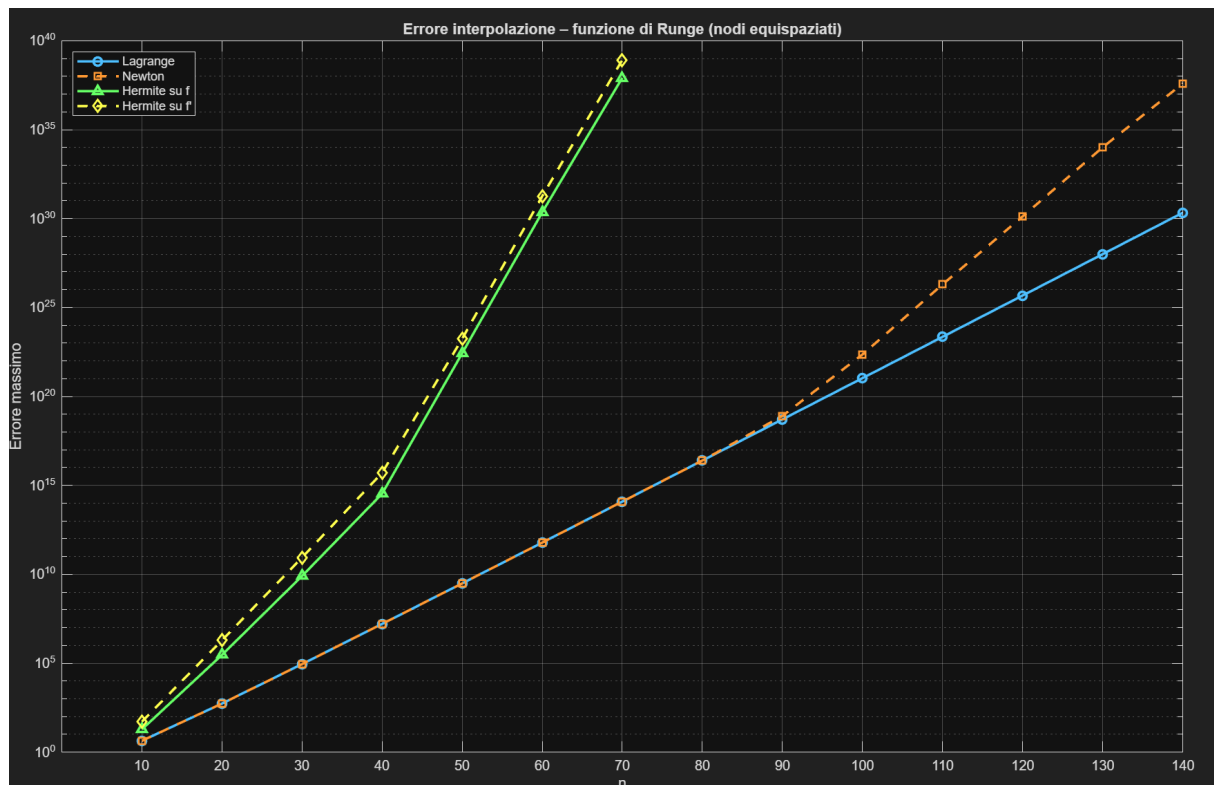
1 function executeInterpolation()
2 % executeInterpolation()
3 % Esegue calcoli di polinomi interpolanti con metodo di Lagrange,
      Newton e
4 % Hermite
5
6 a    = -10;
7 b    = 10;
8 xx   = linspace(a, b, 10001);
9 fxx  = runge(xx);
10 f1xx = drunge(xx);

```

```

11
12 nsLag = 10:10:140;
13 nsHerm = 10:10:70;
14
15 errLag = zeros(size(nsLag));
16 errNew = zeros(size(nsLag));
17 errH = zeros(size(nsHerm));
18 errHy1 = zeros(size(nsHerm));
19
20 for i = 1:length(nsLag)
21     n = nsLag(i);
22     xi = linspace(a, b, n+1);
23     fi = runge(xi);
24     errLag(i) = max(abs(lagrange(xi, fi, xx) - fxx));
25     errNew(i) = max(abs(newton(xi, fi, xx) - fxx));
26 end
27
28 for j = 1:length(nsHerm)
29     n = nsHerm(j);
30     xi = linspace(a, b, n+1);
31     fi = runge(xi);
32     fi1 = drunge(xi);
33     [yh, y1h] = hermite(xi, fi, fi1, xx);
34     errH(j) = max(abs(yh - fxx));
35     errHy1(j) = max(abs(y1h - f1xx));
36 end
37
38 figure(1); clf;
39 semilogy(nsLag, errLag, 'b-o', 'LineWidth', 1.5, 'MarkerSize',
40     5); hold on;
41 semilogy(nsLag, errNew, 'r--s', 'LineWidth', 1.5, 'MarkerSize',
42     5);
43 semilogy(nsHerm, errH, 'm-^', 'LineWidth', 1.5, 'MarkerSize',
44     5);
45 semilogy(nsHerm, errHy1, 'k--d', 'LineWidth', 1.5, 'MarkerSize',
46     5);
47 xlabel('n');
48 ylabel('Errore massimo');
49 title('Errore interpolazione - funzione di Runge (nodi
50     equispaziati)');
51 legend('Lagrange', 'Newton', 'Hermite su f', 'Hermite su f'',
52     ...
53     'Location', 'northwest');
54 grid on;
55 xticks(nsLag);
56 end
57
58 function y = runge(x), y = 1 ./ (1 + x.^2); end
59 function dy = drunge(x), dy = -2*x ./ (1 + x.^2).^2; end

```



Alla luce dei risultati visualizzati nel grafico possiamo trarre le seguenti conclusioni. In linea generale possiamo vedere che con la funzione di Runge interpolata in punti equidistanti, l'interpolazione diverga esponenzialmente al crescere di n . Dal grafico possiamo notare che le curve dei metodi di Newton e Lagrange si sovrappongono, poichè entrambi calcolano il medesimo polinomio. La separazione che avviene per n grande è dovuta dal condizionamento numerico degli algoritmi in virgola mobile.

Con il metodo di Hermite invece si nota che, dovendo interpolare sia la funzione che la sua derivata ottenendo un polinomio di grado $2n + 1$, ha una crescita nettamente maggiore rispetto agli altri metodi. In merito all'errore della derivata prima nel metodo di Hermite possiamo notare inanzitutto che nel grafico la curva della derivata stia sempre sopra alla curva della funzione. Questo accade quando il polinomio oscilla violentemente e di conseguenza la derivata varia ancora più velocemente, amplificando l'errore.

Esercizio 21. Ripetere l'esercizio precedente, utilizzando le ascisse di Chebyshev in luogo di quelle equidistanti. Commentare i risultati ottenuti.

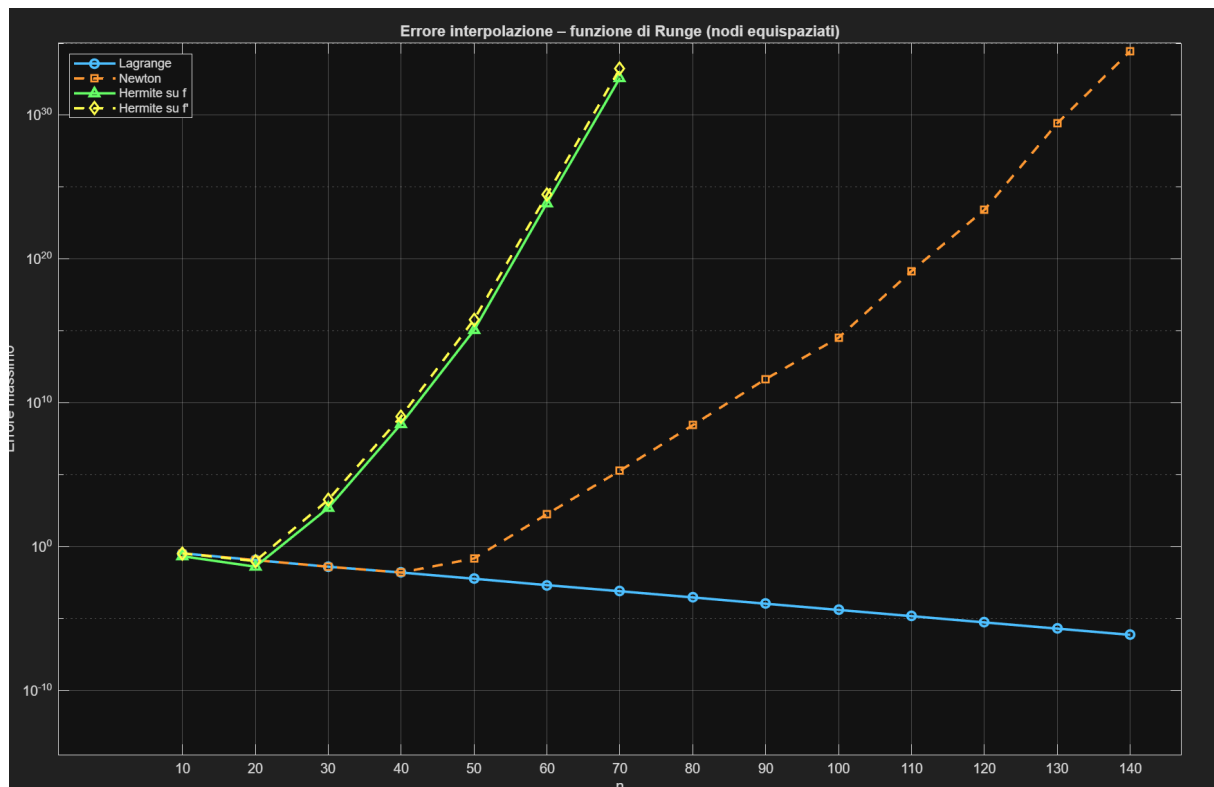
Soluzione:

```
1 function executeInterpolationCheby()
2 % executeInterpolationCheby()
3 % Esegue calcoli di polinomi interpolanti con metodo di Lagrange,
  Newton e
4 % Hermite e ascisse di Chebyshev
5
6 a    = -10;
7 b    = 10;
8 xx   = linspace(a, b, 10001);
```

```

9 fxx = runge(xx);
10 flxx = drunge(xx);
11
12 nsLag = 10:10:140;
13 nsHerm = 10:10:70;
14
15 errLag = zeros(size(nsLag));
16 errNew = zeros(size(nsLag));
17 errH = zeros(size(nsHerm));
18 errHy1 = zeros(size(nsHerm));
19
20 for i = 1:length(nsLag)
21     n = nsLag(i);
22     xi = chebyshev(a, b, n);
23     fi = runge(xi);
24     errLag(i) = max(abs(lagrange(xi, fi, xx) - fxx));
25     errNew(i) = max(abs(newton(xi, fi, xx) - fxx));
26 end
27
28 for j = 1:length(nsHerm)
29     n = nsHerm(j);
30     xi = chebyshev(a, b, n);
31     fi = runge(xi);
32     fi1 = drunge(xi);
33     [yh, y1h] = hermite(xi, fi, fi1, xx);
34     errH(j) = max(abs(yh - fxx));
35     errHy1(j) = max(abs(y1h - flxx));
36 end
37
38 figure(1); clf;
39 semilogy(nsLag, errLag, '-o', 'Color', [0.30 0.75 1.00], '
    LineWidth', 1.8); hold on; % azzurro
40 semilogy(nsLag, errNew, '--s', 'Color', [1.00 0.60 0.20], '
    LineWidth', 1.8); % arancio
41 semilogy(nsHerm, errH, '-~', 'Color', [0.40 1.00 0.40], '
    LineWidth', 1.8); % verde
42 semilogy(nsHerm, errHy1, '--d', 'Color', [1.00 1.00 0.30], '
    LineWidth', 1.8); % giallo
43 xlabel('n');
44 ylabel('Errore massimo');
45 title('Errore interpolazione');
46 legend('Lagrange', 'Newton', 'Hermite su f', 'Hermite su f''',
    ...
    'Location', 'northwest');
47
48 grid on;
49 xticks(nsLag);
50 end
51
52 function y = runge(x), y = 1 ./ (1 + x.^2); end
53 function dy = drunge(x), dy = -2*x ./ (1 + x.^2).^2; end

```



Rispetto al grafico dell'esercizio 21 possiamo notare come l'utilizzo di ascisse di Chebyshev vada a modificare il comportamento dei metodi. Nel metodo di Lagrange l'errore diminuisce al crescere di n , indicando quindi un buon condizionamento del metodo con queste ascisse. Lo stesso discorso non vale però per il metodo di Newton e Hermite, dove invece cominciano a crescere a causa dei limiti della precisione di macchina con l'utilizzo delle differenze divise. Questo perchè cominciando a crescere il valore di n , le differenze divise accumulano errori di arrotondamento poichè si perdono cifre significative.

Esercizio 22. Scrivere una function Matlab, `x = tridia( b, a, c, y )` che risolve un sistema tridiagonale, in cui b , a , c sono i vettori contenenti gli elementi sulle 3 diagonali, e y è un vettore contenente il termine noto.

Soluzione:

```
1 function x = tridia(b, a, c, y)
2 % x = tridia(a,b,c,y)
3 % Metodo di risoluzione di un sistema lineare con matrice
  tridiagonale
4 % fattorizzabile LU
5 % Input
6 %   b, a, c - diagonale inferiore, principale e superiore della
  matrice
7 %   f - vettore dei termini noti
8 % Output
9 %   x - soluzione del sistema lineare
10 if nargin < 4, error("Numero di parametri errato"), end
11 n = length(a);
```

```

12 if length(b) ~= n || length(c) ~= n || length(y) ~= n, error("
    Dati errati"), end
13 errorMsg = "Matrice non fattorizzabile";
14 if a(1) == 0, error(errorMsg), end
15 x = y;
16 for i = 2:n
17     b(i) = b(i) / a(i-1);
18     a(i) = a(i) - b(i)*c(i-1);
19     if a(i) == 0, error(errorMsg), end
20     x(i) = x(i) - b(i) * x(i-1);
21 end
22 x(n) = x(n) / a(n);
23 for i = n-1 : -1: 1
24     x(i) = (x(i)-c(i)*x(i+1)) / a(i);
25 end
26 end

```

Esercizio 23. Dimostrare che, per una spline cubica completa interpolante una funzione $f(x)$ sulla partizione

$$\Delta = \{a = x_0 < x_1 < \dots < x_n = b\}, \quad h_i = x_i - x_{i-1}, \quad i = 1, \dots, n,$$

i valori

$$m_i = s_3''(x_i), \quad i = 0, \dots, n,$$

sono la soluzione del sistema lineare tridiagonale con matrice a diagonale dominante per righe:

$$\begin{pmatrix} 2 & 1 & & & \\ \phi_1 & 2 & \xi_1 & & \\ & \phi_2 & 2 & \xi_2 & \\ & & \ddots & \ddots & \ddots \\ & & & \phi_{n-1} & 2 & \xi_{n-1} \\ & & & & 1 & 2 \end{pmatrix} \begin{pmatrix} m_0 \\ m_1 \\ m_2 \\ \vdots \\ m_{n-1} \\ m_n \end{pmatrix} = 6 \begin{pmatrix} f[x_0, x_0, x_1] \\ f[x_0, x_1, x_2] \\ f[x_1, x_2, x_3] \\ \vdots \\ f[x_{n-2}, x_{n-1}, x_n] \\ f[x_{n-1}, x_n, x_n] \end{pmatrix},$$

con

$$\phi_i = \frac{h_i}{h_i + h_{i+1}}, \quad \xi_i = \frac{h_{i+1}}{h_i + h_{i+1}}, \quad i = 1, \dots, n-1.$$

Soluzione: Sia $s_3(x)$ la spline cubica completa interpolante f sulla partizione Δ . Poiché $s_3(x)$ è una spline cubica per la partizione Δ , $s_3'(x)$ è una spline quadratica e $s_3''(x)$ è una spline lineare su Δ . Pertanto:

$$s_3''(x) = \frac{m_i(x - x_{i-1}) + m_{i-1}(x_i - x)}{h_i}, \quad x \in [x_{i-1}, x_i]$$

Integrando una volta otteniamo:

$$s_3'(x) = \frac{m_i(x - x_{i-1})^2 - m_{i-1}(x_i - x)^2}{2h_i} + q_i, \quad i = 1, \dots, n$$

E integrando ulteriormente otteniamo:

$$s_3(x) = \frac{m_i(x - x_{i-1})^3 + m_{i-1}(x_i - x)^3}{6h_i} + q_i(x - x_{i-1}) + r_i, \quad i = 1, \dots, n$$

Imponendo le condizioni di interpolazione $s_3(x_{i-1}) = f(x_{i-1})$, $s_3(x_i) = f(x_i)$ si ottiene che:

$$r_i = f_{i-1} - m_{i-1} \frac{h_i^2}{6}, \quad q_i = f[x_{i-1}, x_i] - \frac{h_i}{6}(m_i - m_{i-1}), \quad i = 1, \dots, n$$

Per $i = 1, \dots, n-1$ imponiamo la continuità della derivata prima nel nodo x_i , ovvero $s'_3|_{[x_{i-1}, x_i]}(x_i) = s'_3|_{[x_i, x_{i+1}]}(x_i)$:

$$m_i \frac{h_i}{2} + q_i = -m_i \frac{h_{i+1}}{2} + q_{i+1}$$

Sostituendo le espressioni di q_i, q_{i+1} e riordinando si ottiene:

$$\frac{m_{i-1}}{6}h_i + \frac{m_i}{6}2(h_i + h_{i+1}) + \frac{m_{i+1}}{6}h_{i+1} = f[x_i, x_{i+1}] - f[x_{i-1}, x_i]$$

Moltiplicando membro a membro per 6 e dividendo per $h_i + h_{i+1} = x_i - x_{i-1} + x_{i+1} - x_i = x_{i+1} - x_{i-1}$:

$$\begin{aligned} m_{i-1} \frac{h_i}{h_i + h_{i+1}} + 2m_i + m_{i+1} \frac{h_{i+1}}{h_i + h_{i+1}} &= 6f[x_{i-1}, x_i, x_{i+1}] \\ \implies \phi_i m_{i-1} + 2m_i + \xi_i m_{i+1} &= 6f[x_{i-1}, x_i, x_{i+1}] \end{aligned}$$

con $\phi_i = \frac{h_i}{h_i + h_{i+1}}$, $\xi_i = \frac{h_{i+1}}{h_i + h_{i+1}}$, e $\phi_i + \xi_i = 1$. Queste sono le $n-1$ equazioni interne.

La spline completa impone inoltre le condizioni $s'_3(x_0) = f'(x_0)$ e $s'_3(x_n) = f'(x_n)$.

Estremo sinistro. Valutando $s'_3(x)$ sull'intervallo $[x_0, x_1]$ in $x = x_0$:

$$s'_3(x_0) = \frac{-m_0 h_1^2}{2h_1} + q_1 = -\frac{h_1}{3}m_0 - \frac{h_1}{6}m_1 + f[x_0, x_1]$$

Imponendo $s'_3(x_0) = f'(x_0)$ e moltiplicando per $-6/h_1$:

$$2m_0 + m_1 = \frac{6(f[x_0, x_1] - f'(x_0))}{h_1}$$

Poiché $f[x_0, x_0] = f'(x_0)$, la differenza divisa con nodo ripetuto vale

$$f[x_0, x_0, x_1] = \frac{f[x_0, x_1] - f[x_0, x_0]}{x_1 - x_0} = \frac{f[x_0, x_1] - f'(x_0)}{h_1},$$

da cui

$$2m_0 + m_1 = 6f[x_0, x_0, x_1]$$

Estremo destro. In modo simmetrico, valutando $s'_3(x)$ sull'intervallo $[x_{n-1}, x_n]$ in $x = x_n$:

$$s'_3(x_n) = \frac{h_n}{3}m_n + \frac{h_n}{6}m_{n-1} + f[x_{n-1}, x_n]$$

Imponendo $s'_3(x_n) = f'(x_n)$, moltiplicando per $6/h_n$ e usando $f[x_n, x_n] = f'(x_n)$:

$$m_{n-1} + 2m_n = \frac{6(f'(x_n) - f[x_{n-1}, x_n])}{h_n} = 6f[x_{n-1}, x_n, x_n]$$

Le equazioni interne e quelle al contorno formano il sistema tridiagonale $(n+1) \times (n+1)$:

$$\begin{pmatrix} 2 & 1 & & & \\ \phi_1 & 2 & \xi_1 & & \\ & \phi_2 & 2 & \xi_2 & \\ & & \ddots & \ddots & \ddots \\ & & & \phi_{n-1} & 2 & \xi_{n-1} \\ & & & & 1 & 2 \end{pmatrix} \begin{pmatrix} m_0 \\ m_1 \\ m_2 \\ \vdots \\ m_{n-1} \\ m_n \end{pmatrix} = 6 \begin{pmatrix} f[x_0, x_0, x_1] \\ f[x_0, x_1, x_2] \\ f[x_1, x_2, x_3] \\ \vdots \\ f[x_{n-2}, x_{n-1}, x_n] \\ f[x_{n-1}, x_n, x_n] \end{pmatrix}$$

che è la tesi.

Infine, sulle righe interne $|2| > \phi_i + \xi_i = 1$ e sulle due righe di bordo $|2| > |1|$: la matrice è quindi strettamente dominante diagonale per righe, dunque non singolare. Il sistema ammette un'unica soluzione $(m_0, \dots, m_n)$ ed è fattorizzabile LU senza pivoting con costo lineare; tali momenti individuano univocamente la spline cubica completa interpolante. $\square$

Esercizio 24. Utilizzando la function del precedente esercizio 22, scrivere la function Matlab, `y = spline3( xi, fi, x, type, f1a, f1b )` in cui (xi, fi) sono i dati di interpolazione e x 'e un vettore (riga o colonna), contenente i punti in cui valutare la spline cubica interpolante, restituiti nel vettore y, che avr'a le stesse dimensioni di x. Infine, type 'e un flag che vale 1 (valore di default) per la spline cubica not-a-knot, 0 per quella naturale, 2 per la spline cubica completa. In quest'ultimo caso, gli ultimi due parametri di ingresso (opzionali) contengono il valore della derivata della funzione interpolante negli estremi.

Soluzione:

```
1 function y = spline3(xi, fi, x, type, f1a, f1b)
2 % y = spline3(xi, fi, x, type, f1a, f1b)
3 % Spline cubica interpolante via metodo dei momenti (sistema
  tridiagonale)
4 % Input:
5 %   xi - ascisse di interpolazione
6 %   fi - valori della funzione nei punti xi
7 %   x - punti dove valutare la spline
8 %   type - 0 naturale, 1 not-a-knot (default), 2 completa
9 %   f1a, f1b - derivate prime agli estremi (richieste solo se
  type==2)
10 % Output:
11 %   y - valutazioni della spline (stessa forma di x)
12
13 if nargin < 3, error('Parametri insufficienti'), end
14 if nargin < 4, type = 1; end
15 if type == 2 && nargin < 6, error('Per la spline completa servono
  f1a e f1b'), end
```

```

16
17 xi = xi(:);
18 fi = fi(:);
19 n = length(xi) - 1;
20 if n + 1 ~= length(fi), error('Lunghezze dei vettori in ingresso
    non valide'), end
21 if n < 2, error('Servono almeno 3 nodi'), end
22 if type == 1 && n < 3, error('La spline not-a-knot richiede
    almeno 4 nodi'), end
23
24 h = diff(xi);
25 delta = diff(fi) ./ h;
26
27 phi = zeros(n+1,1); ksi = zeros(n+1,1); dd = zeros(n+1,1);
28 for k = 2:n
29     hl = h(k-1); hr = h(k);
30     phi(k) = hl / (hl + hr);
31     ksi(k) = hr / (hl + hr);
32     dd(k) = 6*(delta(k) - delta(k-1)) / (xi(k+1) - xi(k-1));
33 end
34
35 m = zeros(n+1, 1);
36
37 if type == 0 || type == 2
38     b = zeros(n+1,1); a = zeros(n+1,1); c = zeros(n+1,1); rhs
    = zeros(n+1,1);
39     a(2:n) = 2;
40     b(2:n) = phi(2:n);
41     c(2:n) = ksi(2:n);
42     rhs(2:n) = dd(2:n);
43     if type == 0 % NATURALE
44         a(1) = 1; c(1) = 0; rhs(1) = 0;
45         a(n+1) = 1; b(n+1) = 0; rhs(n+1) = 0;
46     else % COMPLETA
47         a(1) = 2; c(1) = 1; rhs(1) = 6*(delta(1) - f1a) / h(1);
48         a(n+1) = 2; b(n+1) = 1; rhs(n+1) = 6*(f1b - delta(n)) /
    h(n);
49     end
50     m = tridia(b, a, c, rhs);
51
52 else % NOT-A-KNOT
53     b = zeros(n-1,1); a = zeros(n-1,1); c = zeros(n-1,1); rhs
    = zeros(n-1,1);
54     for j = 1:n-1
55         k = j + 1;
56         b(j) = phi(k); a(j) = 2; c(j) = ksi(k); rhs(j) = dd(k)
    ;
57     end
58     a(1) = 2 - phi(2); c(1) = ksi(2) - phi(2); rhs(1) = ksi(2)
    * dd(2);
59     b(n-1) = phi(n) - ksi(n); a(n-1) = 2 - ksi(n); rhs(n-1) =

```

```

        phi(n) * dd(n);
60
61     mInt = tridia(b, a, c, rhs);
62     m(2:n) = mInt;
63     m(1) = dd(2) - m(2) - m(3);
64     m(n+1) = dd(n) - m(n-1) - m(n);
65 end
66
67 y = zeros(size(x));
68 for j = 1:length(x)
69     if x(j) <= xi(1)
70         i = 1;
71     elseif x(j) >= xi(n+1)
72         i = n;
73     else
74         i = find(xi <= x(j), 1, 'last');
75         if i > n, i = n; end
76     end
77     d1 = xi(i+1) - x(j);
78     d0 = x(j) - xi(i);
79     y(j) = (m(i)*d1^3 + m(i+1)*d0^3)/(6*h(i)) ...
80           + (fi(i)/h(i) - h(i)*m(i)/6) * d1 ...
81           + (fi(i+1)/h(i) - h(i)*m(i+1)/6) * d0;
82 end
83 end

```

Esercizio 25. Utilizzare la function `spline3` per approssimare la funzione di Runge, come nell'esercizio 20, utilizzando $n + 1$ ascisse equidistanti, con $n = 100, 200, \dots, 2000$. Detta $h = \frac{20}{n}$ la distanza tra due punti consecutivi, graficare, in formato `loglog` l'errore massimo, calcolato come nell'esercizio 20, rispetto ad h . Con che potenza di h decresce, asintoticamente l'errore, per le tre spline interpolanti?

Soluzione:

```

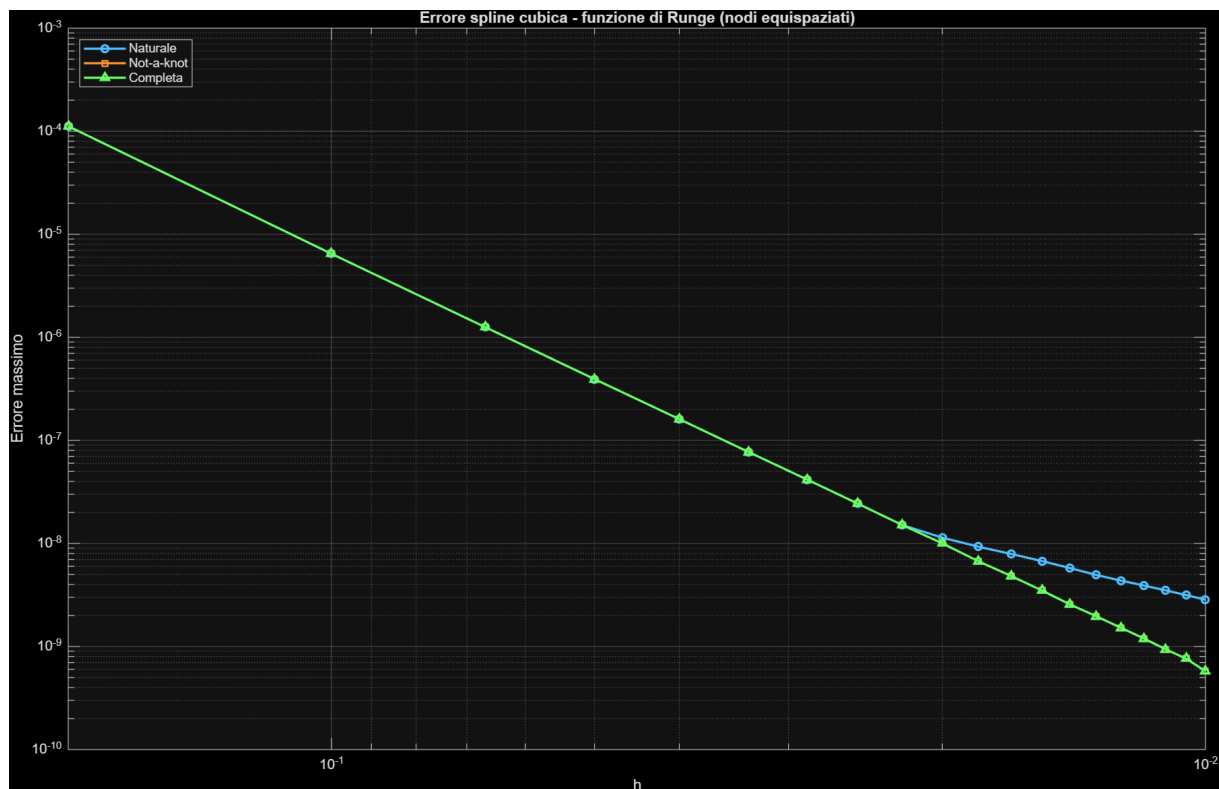
1 function executeSpline3()
2 % executeSpline3()
3 % Metodo di esecuzione della function spline3
4
5 a = -10;
6 b = 10;
7 xx = linspace(a, b, 10001);
8 fxx = runge(xx);
9
10 ns = 100:100:2000;
11 h = 20 ./ ns;
12
13 errNat = zeros(size(ns));
14 errKnot = zeros(size(ns));
15 errComp = zeros(size(ns));
16

```

```

17 for i = 1:length(ns)
18     n = ns(i);
19     xi = linspace(a, b, n+1);
20     fi = runge(xi);
21
22     yNat = spline3(xi, fi, xx, 0); % naturale
23     yKnot = spline3(xi, fi, xx, 1); % not-a-knot
24     yComp = spline3(xi, fi, xx, 2, drunge(a), drunge(b)); %
    completa
25
26     errNat(i) = max(abs(yNat - fxx));
27     errKnot(i) = max(abs(yKnot - fxx));
28     errComp(i) = max(abs(yComp - fxx));
29 end
30
31 % Grafico loglog
32 figure(1); clf;
33 loglog(h, errNat, '-o', 'Color', [0.30, 0.75, 1.00], 'LineWidth',
    1.6, 'MarkerSize', 5); hold on;
34 loglog(h, errKnot, '-s', 'Color', [1.00 0.60 0.20], 'LineWidth',
    1.6, 'MarkerSize', 5);
35 loglog(h, errComp, '-^', 'Color', [0.40 1.00 0.40], 'LineWidth',
    1.6, 'MarkerSize', 5);
36
37 xlabel('h'); ylabel('Errore massimo');
38 title('Errore spline cubica - funzione di Runge (nodi
    equispaziati)');
39 legend('Naturale', 'Not-a-knot', 'Completa', ...
    'pendenza h^2', 'pendenza h^4', 'Location', 'northwest');
40
41 grid on;
42 end
43
44 % Funzioni ausiliarie
45 function y = runge(x), y = 1 ./ (1 + x.^2); end
46 function dy = drunge(x), dy = -2*x ./ (1 + x.^2).^2; end

```



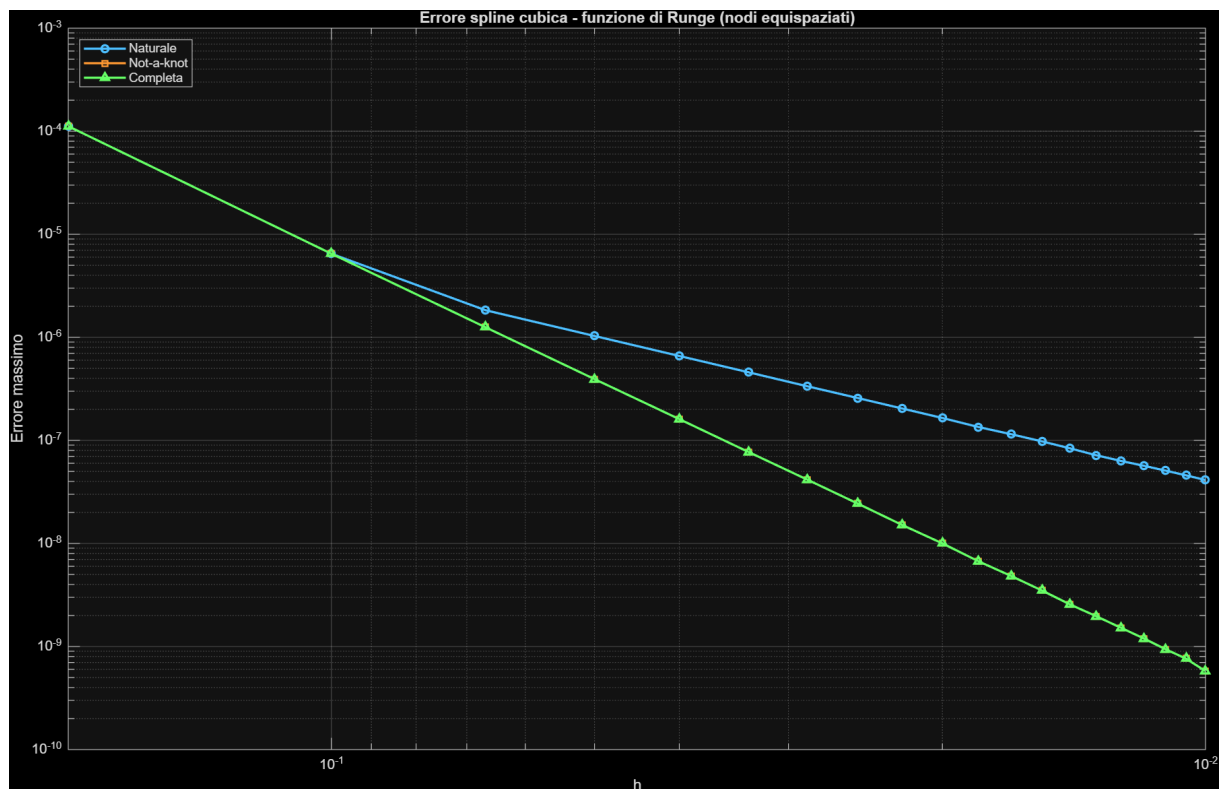
Alla luce del grafico possiamo notare che alla crescita di n , decresce il passo $h = \frac{20}{n}$ da 10^{-1} a 10^{-2} . Vediamo inoltre l'andamento dell'errore delle spline cubiche, che in tutti i casi diminuisce in maniera costante.

In particolare la spline cubica completa e not-a-knot coincidono tra di loro con una decrescita dell'ordine di $O(h^4)$. Questo avviene perchè entrambe mantengono accuratezza anche ai bordi (la completa tramite la conoscenza esatta delle derivate, la not-a-knot tramite le formule per ricavare i suoi estremi).

Nel caso della spline cubica naturale invece abbiamo una decrescita dell'ordine di $O(h^2)$. Questo è dovuto dall'imposizione della derivata seconda nulla agli estremi, che non coincide con il vero valore della derivata seconda della funzione presa in esame.

Esercizio 26. Ripetere il precedente esercizio considerando, invece dell'intervallo $[-10, 10]$, l'intervallo $[-5, 15]$. Commentare i risultati ottenuti, rispetto a quelli del precedente esercizio.

Soluzione:



Come nel grafico precedente, la decrescita di $h = \frac{20}{n}$ è la medesima perchè l'ampiezza dell'intervallo è sempre 20.

Rimangono invariati anche i comportamenti delle tre spline, anche in merito all'ordine di decrescita che rimangono $O(h^4)$ per le spline completa e not-a-knot e $O(h^2)$ per la spline naturale.

La differenza evidente è la pendenza della curva della spline naturale. Questo è dovuto sempre dal problema riguardo l'approssimazione della derivata seconda, che nell'intervallo $[-5, 15]$ è maggiore.

Esercizio 27. Costruire una function Matlab,

```
ci = ncotes( n )
```

che calcoli i coefficienti della formula di Newton-Cotes di grado n. Costruire una tabella con tutti i coefficienti delle formule ammissibili per ragioni di condizionamento. I coefficienti della tabella devono essere riportati come numeri razionali.

Soluzione:

```
1 function ci = ncotes(n)
2 % ci = ncotes(n)
3 % Metodo di calcolo dei coefficienti della formula di Newton-
  Cotes di
4 % grado n
5 % Input:
6 %   n - grado della formula
7 % Output:
8 %   ci - vettore coefficienti di grado n
```

```

9 if nargin < 1, error('Numero parametri insufficiente'), end
10 if n <= 0, error('Grado della formula non valido'), end
11
12 ci = zeros(n+1, 1);
13 for i = 0:n
14     ind = [0:i-1, i+1:n];
15     din = prod(i-ind);
16     a = poly(ind);
17     q = [a./(n+1:-1:1), 0];
18     alfain = polyval(q, n);
19     ci(i+1) = alfain / din;
20     format rat;
21 end
22 end

```

```

>> for n = 1:7
    fprintf('n = %d: %s\n', n, strtrim(rats(ncotes(n).')));
end
n = 1: 1/2      1/2
n = 2: 1/3      4/3      1/3
n = 3: 3/8      9/8      9/8      3/8
n = 4: 14/45    64/45    8/15    64/45    14/45
n = 5: 95/288   125/96    125/144  125/144  125/96    95/288
n = 6: 41/140   54/35     27/140  68/35    27/140    54/35    41/140
n = 7: 108/355  810/559   343/640  649/536  649/536   343/640   810/559   108/355
>>

```

Esercizio 28. Costruire una function Matlab,

`[Ikn,err] = ncotescomp( fx, a, b, k, n )`

che calcoli l'approssimazione dell'integrale della funzione f_x sull'intervallo $[a,b]$, utilizzando la formula di Newton-Cotes di grado k su $n + 1$ ascisse equidistanti, con n multiplo pari di k , insieme ad una stima dell'errore di quadratura.

Soluzione:

```

1 function [Ikn, err] = ncotescomp(fx, a, b, k, n)
2 % [Ikr, err] = ncotescomp(fx, a, b, k, n)
3 % Metodo di calcolo dell'approssimazione dell'integrale di una
   funzione f
4 % nell'intervallo [a, b] tramite la formula composta di Newton-
   Cotes
5 % Input:
6 %   fx - function che implementa la funzione f
7 %   a, b - estremi di integrazione
8 %   k - grado della formula di Newton-Cotes
9 %   n - numero di suddivisioni dell'intervallo
10 % Output:
11 %   Ikn - approssimazione dell'integrale
12 %   err - stima dell'errore di quadratura
13 if nargin < 5, error('Numero parametri insufficiente'), end
14 if a >= b, error('Intervallo di integraizone non valido'), end

```

```

15 if k <= 0 || k ~= fix(k), error('Grado della formula non valido')
    , end
16 if n <= 0 || mod(n, k) ~= 0 || mod(n/k, 2) ~= 0, error('Numero di
    suddivisioni dell''intervallo non valido'), end
17
18 h = (b - a) / n;
19 x = linspace(a, b, n+1);
20 f = feval(fx, x);
21 f = f(:);
22 ci = ncotes(k);
23 ci = ci(:);
24
25 Ikn = 0;
26 for i = 1:n/k
27     idx = (i-1)*k+1:i*k+1;
28     Ikn = Ikn + h * sum(ci .* f(idx));
29 end
30
31 Ikn2 = 0;
32 for i = 1:n/(2*k)
33     idx = (i-1)*2*k+1:2:i*2*k+1;
34     Ikn2 = Ikn2 + (2*h) * sum(ci .* f(idx));
35 end
36
37 if mod(k, 2) == 0
38     p = k+2;
39 else
40     p = k+1;
41 end
42
43 err = (Ikn - Ikn2) / (2^p - 1);
44 end

```

Esercizio 29. Scrivere una function Matlab che calcoli l'approssimazione di un integrale definito mediante la formula adattiva di Simpson con una tolleranza prescritta tol.

Soluzione:

```

1 function I2 = simpAdapt(f, a, b, tol, fa, fb, fc)
2 % I2 = simpAdapt(f, a, b, tol, fa, fb, fc)
3 % Metodo di calcolo dell'integrale di una funzione f per
    intervallo [a, b]
4 % tramite la formula adattiva di Simpson
5 if nargin < 4, error('Numero parametri insufficiente'), end
6 if nargin < 7, fc = feval(f, (a+b)/2); end
7 if nargin < 6, fb = feval(f, b); end
8 if nargin < 5, fa = feval(f, a); end
9 if a >= b, error('Estremi dell''intervallo non validi'), end
10 if tol <= 0, error('Tolleranza non valida'), end

```



```

11
12 h = (b-a)/2;
13 I1 = (h/3) * (fa+4*fc+fb);
14
15 c = (a+b)/2;
16 d = (a+c)/2;
17 e = (c+b)/2;
18 fd = feval(f, d);
19 fe = feval(f, e);
20
21 I2 = (h/6) * (fa + 4*fd + 2*fc + 4*fe + fb);
22 err = abs(I2 - I1)/15;
23
24 if err > tol
25     I2 = simpAdapt(f, a, c, tol / 2, fa, fc, fd) + ...
26         simpAdapt(f, c, b, tol / 2, fc, fb, fe);
27 end
28 end

```

Esercizio 30. Utilizzare le function dei precedenti due esercizi per approssimare

$$\int_0^n 10^2 \sin(10^2 x) e^{-10^2 x} dx \approx \frac{1}{2}$$

con la formula composta, e quella adattativa, di Simpson. Per la formula composta, tabulare l'errore e la sua stima, per $n = 40, 80, 160, 320$. Per la formula adattativa, tabulare l'errore e tol, per $\text{tol} = 10^{-3}, 10^{-4}, 10^{-5}, 10^{-6}$. In entrambi i casi, riportare il numero di valutazioni funzionali effettuate.

Soluzione:

```

1 function executeQuadrature()
2 % executeQuadrature()
3 % Approssima int_0^1 100*sin(100x)*exp(-100x) dx (~ 1/2) con
  la formula
4 % composta di Newton-Cotes e adattiva di Simpson, con
  tabulazione dei
5 % dati.
6
7 f = @(x) 100*sin(100*x).*exp(-100*x);
8 a = 0;
9 b = 1;
10 Iexact = 0.5;
11
12 nfeval = 0;
13 g = @count;
14
15 % ---- formula composta di Newton-Cotes ----
16 fprintf('Formula composta di Newton-Cotes \n');
17 fprintf('%6s %12s %18s %14s %14s\n', 'n', 'val.funz.', 'I (approx

```

```

    ), 'errore', 'stima err.');
```

```

18 for n = [40 80 160 320]
19     nfeval = 0;
20     [Ikn, err] = ncotescomp(g, a, b, 2, n);
21     fprintf('%6d %12d %18.12f %14.3e %14.3e\n', n, nfeval, Ikn,
        abs(Ikn - Iexact), abs(err));
22 end
23
24 % ---- formula adattiva di Simpson ----
25 fprintf('\nFormula adattiva di Simpson\n');
26 fprintf('%8s %12s %18s %14s\n', 'tol', 'val.funz.', 'I (approx)',
        'errore');
```

```

27 for tol = [1e-3 1e-4 1e-5 1e-6]
28     nfeval = 0;
29     I = simpAdapt(g, a, b, tol);
30     fprintf('%8.0e %12d %18.12f %14.3e\n', tol, nfeval, I, abs(I
        - Iexact));
31 end
32
33 % funzione annidata: valuta f e aggiorna il contatore di
executeQuadrature
34 function y = count(x)
35     nfeval = nfeval + numel(x);
36     y = f(x);
37 end
38 end

```

Tabella Formula composta di Newton-Cotes

n	Val. funz	I (approx)	Errore	Stima err.
40	41	0.154670890620	3.453×10^{-01}	1.319×10^{-2}
80	81	0.466748873084	3.325×10^{-02}	2.081×10^{-2}
160	161	0.498160774812	1.839×10^{-03}	2.094×10^{-3}
320	321	0.499891623887	1.084×10^{-04}	1.154×10^{-4}

Tabella Formula adattiva di Simpson

tol	Val. funz	I (approx)	Errore
1×10^{-3}	5	-0.000000000061	5.000×10^{-01}
1×10^{-4}	5	-0.000000000061	5.000×10^{-01}
1×10^{-5}	5	-0.000000000061	5.000×10^{-01}
1×10^{-6}	5	-0.000000000061	5.000×10^{-01}